V1 RETROSPECTIVE · OCTOBER 2026

From “I need a video...” to one command.

Framewise helps people start with the result they want, then makes FFmpeg feel like a series of small, understandable choices.

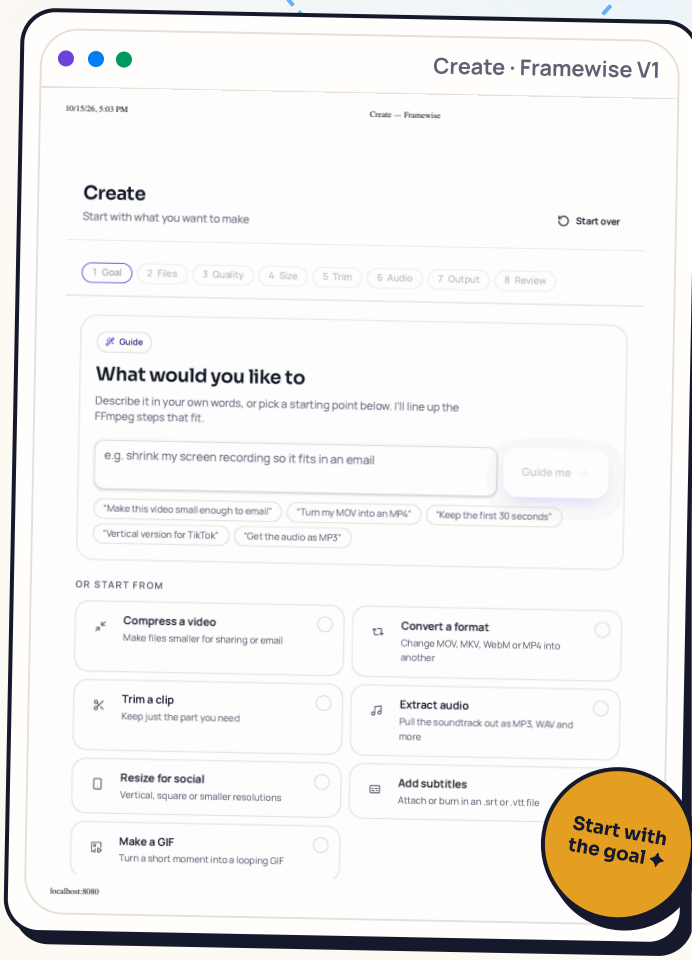
Follow the build story ↓

See V1 in action

7 guided workflows

Browser + desktop

Local run history



A LITTLE TIMELINE

A prototype became a project we could keep building.

Six small milestones moved Framewise from a visual-builder idea to a working local app with a clear source of truth.

01
OCT 12

The first Lovable prototype

Create, History, and Help set the direction: plain language up front, FFmpeg detail when it helps.

02
OCT 12–13

Choose one home for the code

The Lovable export stayed separate. The private `framewise` repository became the master project.

03
OCT 13

Write down what was missing

A project-status report and next steps made the app easier to continue outside its original builder.

04
OCT 13–14

Keep V1's "AI" small and local

Keyword matching was enough for a first guide. No model API, account, or secret key was needed.

05
OCT 14–15

Make commands and setup clearer

Folder-aware desktop paths, more useful unknown-request guidance, and official FFmpeg references improved the handoff.

06
OCT 15

Run the first-use walkthrough

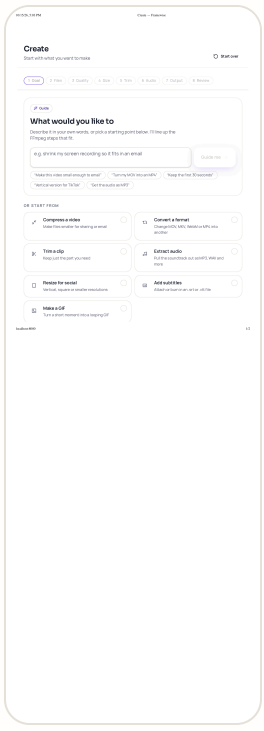
Resize, a two-file batch, folder input/output, Help, and History repeat were walked through manually.

A PEEK INSIDE V1

Three screens. One friendly path through the work.

These are browser-print captures from the local V1 app, made October 15, 2026.

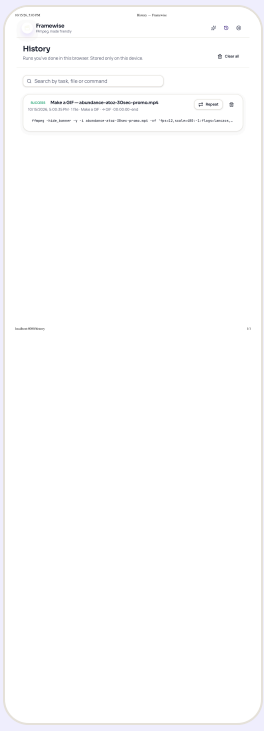
The captures show the product as it was tested locally; they are not screenshots of a hosted production release.



01 · CREATE

Start with the outcome.

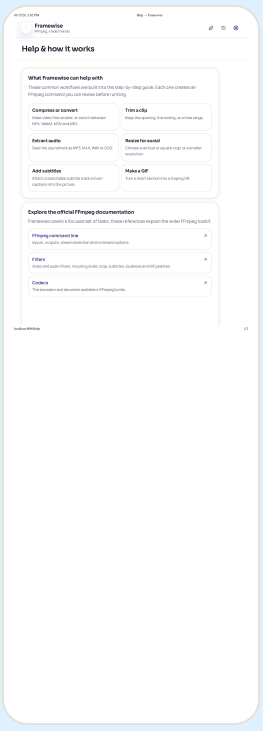
Describe it in your own words, or pick one of seven familiar starting points.



02 · HISTORY

Pick up where you left off.

Saved settings can be repeated; files are selected again for privacy.



03 · HELP

Know what it can do.

Get the supported subset, browser limits, and a path to the wider FFmpeg toolkit.

UNDER THE HOOD

A clear path from “what if?” to “done.”

V1’s helper is intentionally modest: it matches words locally to supported workflows and presets. Then the user stays in control of each next step.

1. **Describe**
   Say what you want to make.
2. **Choose**
   Pick or skip the available steps.
3. **Review**
   See the command and its meaning.
4. **Run**
   Use browser WebAssembly or desktop FFmpeg.
5. **Repeat**
   Keep settings in local browser history.

THINGS V1 TAUGHT US

A little guidance can make a technical tool feel human.

Start with the result

People can explain what they want without knowing FFmpeg flags. Keep goals in everyday language.

Reveal details at the right time

Optional steps lower the barrier, but repeated copy can still slow the flow. V2 can say less, sooner.

Show what will happen

A readable summary plus a full, copyable command helps the user trust the result and stay in control.

Batch needs a clear status

Show the file count, shared settings, destination, and each file's progress—not just a multi-select.

✦ **Evidence note:** the first-use walkthrough was a manual check, not a full browser, codec, file-size, or compatibility matrix. There were no formal usability sessions or analytics.

WHERE V2 COULD GO

Less reading. Same confidence.

These are design hypotheses to explore, not research findings.

01

Lead with a few outcome tiles

"Make smaller," "Change format," and "Edit a clip" can help a first-time user find the right path faster.

02

Keep explanations close to the choice

Use a short sentence in the flow and reveal "Why this?" details when someone wants them.

03

Make batch mode unmistakable

Keep the shared settings, file count, and destination visible as the queue moves.

04

Explain the first engine download

Set expectations before the first run, then show progress and a useful recovery step.

A useful next session: ask first-time users to compress a clip, make a vertical version, and process a folder. Watch for task choice, command confidence, saved output, and repeatability.

THE V1 TOOLKIT

A browser app, with a focused media engine.

No cloud account or application database is required for the V1 workflow.

INTERFACE

React + TypeScript

Typed UI and browser behavior for the guided workflow.

React 19 · TypeScript

APP FRAMEWORK

TanStack Start

App routes for Create, History, and Help.

TanStack Start · TanStack Router

DEVELOPER WORKFLOW

Vite + Nitro

Local development and a production build; Cloudflare output is configured, not deployed.

Vite 8 · Nitro

LOOK AND CONTROLS

Tailwind + Radix

Responsive styling and accessible UI primitives, with Lucide icons.

Tailwind CSS 4 · Radix UI · Lucide

MEDIA PROCESSING

FFmpeg WebAssembly

Runs conversions in the browser tab. The engine is fetched from a public CDN on first use.

@ffmpeg/ffmpeg · @ffmpeg/util

LOCAL BY DESIGN

Browser APIs + storage

File selection and outputs use browser APIs. Run settings stay in localStorage on this device.

File APIs · localStorage · No app database

WANT TO TRY IT LOCALLY?

A laptop, Node.js, and the project folder are enough.

From the Framewise repository root, install the app packages and start the local development server:

```
npm install
npm run dev
```

Browser processing: Framewise uses FFmpeg WebAssembly. You do not need to install native FFmpeg to try conversions in the app.

Copied desktop commands: install FFmpeg on your computer if you want to run a generated command in Terminal or a shell.

HELPFUL PLACES TO START

Official docs & downloads

Links open the maintainers' own documentation.

LOCAL SETUP AND MEDIA TOOLS

- **Node.js downloads** — npm is included with Node.js
- **Install Node.js + npm** — Official npm setup guide
- **Git downloads** — Clone the Framewise repository
- **Get desktop FFmpeg** — Official download page and OS builds
- **FFmpeg WebAssembly** — Browser engine install guide
- **Framewise source** — Master project repository

FFMPEG COMMAND REFERENCES

- **Command line** — Inputs, outputs, options, and streams
- **Filters** — Scale, crop, subtitles, and more
- **Codecs** — Available encoders and decoders
- **Formats** — Containers and media formats

FRAMEWORKS AND UI TOOLS

- **React** — Official learning guide
- **TanStack Start** — React framework docs
- **Vite** — Development and build guide
- **TypeScript** — Install and compiler references
- **Tailwind CSS** — Install and styling docs
- **Radix UI** — Accessible UI primitives
- **Lucide** — Icons used in the interface